FIFO Controller Design Report

From Synchronous to Asynchronous Architecture

Keywords: FIFO, Clock Domain Crossing, Gray Code, Metastability, RTL Design

Xuanhao Zhang

School of Electronic Science and Engineering

Jilin University

Source Code: github.com/xuanhaozhang883/FIFO-project

(Email:3876106498@qq.com)

April 18, 2026

Abstract

This report presents the complete design and verification of a First-In First-Out (FIFO) buffer, progressing from a synchronous single-clock implementation to a fully functional asynchronous dual-clock design. This project covers three parts: (1) a parameterizable synchronous FIFO with full/empty flag generation based on a $(N+1)$-bit pointer scheme; (2) an asynchronous FIFO employing Gray-code pointer synchronization across clock domain crossings (CDC); and (3) a SystemVerilog testbench with self-checking reference model for functional verification. All designs are implemented in RTL Verilog/SystemVerilog and validated via simulation.

Contents

1 Introduction **3**

1.1 Motivation and Background 3

1.2 Project Objectives 3

1.3 Report Organization 3

2 Design Specifications **3**

2.1 Interface Ports 4

2.2 Parameters 4

2.3 Performance Targets 5

3 Synchronous FIFO Architecture **5**

3.1 Overall Architecture 5
3.2 State Detection Strategy Based on $(N+1)$-bit Pointers 5
3.3 Handling Non-Power-of-Two Depth via Precise Pointer Rollover 6
3.4 Simulation Results 6

4 Asynchronous FIFO Architecture **7**
4.1 CDC Challenges and Metastability 7
4.2 Gray Code Encoding Strategy 7
4.3 Two-Stage Synchronizer 8
4.4 Pessimistic Flag Assertion and Timing Safety 8
4.5 Design Decision Matrix 9
4.6 Simulation Results 10

5 Verification Methodology **10**
5.1 Overview of Verification Strategy 10
5.2 Reference Model 10
5.3 Randomized Stimulus Generation 11
5.4 Self-checking and Timing-aware Verification 12
5.5 Test Scenarios 12

6 Challenges and Debugging **13**
6.1 Issue 1: Almost-Full False Alarm in Asynchronous FIFO Testbench 13
6.2 Issue 2: Pointer Wrap-Around Error in Synchronous FIFO 13
6.3 Lessons Learned 14

7 Comparative Analysis: Sync vs. Async FIFO **14**

8 Conclusion **14**

9 Future Improvements **15**

1 Introduction

1.1 Motivation and Background

Modern SoC designs routinely interface subsystems operating at different frequencies. A FIFO buffer serves as the canonical solution for rate-matching and data integrity across such boundaries. This project systematically develops FIFO designs in two stages:

1. **Stage I — Synchronous FIFO**: establishes the foundational pointer and flag logic within a single clock domain.
2. **Stage II — Asynchronous FIFO**: extends the design to handle CDC safely, introducing Gray-code encoding and two-stage synchronizers.

Beyond RTL implementation, this project emphasizes a rigorous verification methodology, utilizing a SystemVerilog-based self-checking testbench to guarantee functional correctness under extreme frequency ratios.

1.2 Project Objectives

- **Design Scalability**: Implement highly parameterizable FIFO architectures to support flexible `DATA_WIDTH` and `DEPTH` configurations, enhancing hardware IP reuse.
- **CDC Resiliency**: Ensure metastability-safe Clock Domain Crossing (CDC) via Gray-code pointer synchronization and two-stage synchronizers.
- **Automated Verification**: Develop a self-checking SystemVerilog testbench utilizing a dynamic-queue-based golden model for 100% automated data consistency checks.
- **Corner-Case Validation**: Verify system stability and data integrity under extreme conditions, such as simultaneous operations at full/empty boundaries and high frequency ratios.

1.3 Report Organization

The remainder of this report is structured as follows. **Section 2** defines the hardware specifications and interface requirements; **Section 3** and **Section 4** explore the architectural evolution from a foundational synchronous design to a metastability-resilient asynchronous FIFO utilizing Gray-code synchronization; **Section 5** details the verification methodology using a self-checking SystemVerilog environment ; while **Section 6** and **Section 7** provide critical debugging insights and a comparative analysis between synchronous and asynchronous architectures; **Section 8** concludes the report with a summary of achievements and future optimization goals.

2 Design Specifications

2.1 Interface Ports

Table 1 Common Port Descriptions (Synchronous FIFO)

Port	Direction	Width	Description
`clk`	Input	1	System clock (rising edge active)
`reset`	Input	1	Asynchronous reset, active-high
`wr_en`	Input	1	Write enable
`rd_en`	Input	1	Read enable
`data_in`	Input	`DATA_WIDTH`	Data input bus
`data_out`	Output	`DATA_WIDTH`	Data output bus
`full`	Output	1	FIFO full flag
`empty`	Output	1	FIFO empty flag

Table 2 Additional Ports for Asynchronous FIFO

Port	Direction	Width	Description
`wr_clk`	Input	1	Write-domain clock
`rd_clk`	Input	1	Read-domain clock
`almost_full`	Output	1	FIFO almost full flag
`almost_empty`	Output	1	FIFO almost empty flag

2.2 Parameters

Table 3 Configurable Parameters

Parameter	Default	Range	Description
`DATA_WIDTH`	8	[4, 128]	Width of the data bus in bits
`DEPTH`	16	2^N	Number of words in FIFO
`ALMOST_FULL_TH`	4	$[1, DEPTH - 1]$	Threshold for early full
`ALMOST_EMPTY_TH`	4	$[1, DEPTH - 1]$	Threshold for early empty

Key Insight

While synchronous designs can utilize arbitrary DEPTH with custom wrap-around logic to prevent invalid addresses, asynchronous FIFO strictly requires 2^N depths. This ensures Gray-code continuity; any other wrap-around point would trigger multi-bit transitions, violating the single-bit change requirement necessary for safe CDC.

2.3 Performance Targets

- Target clock frequency: 200 MHz (synchronous), 100 MHz write / 80 MHz read (asynchronous).
- Achieves data availability within 3–4 clock cycles in the asynchronous design, accounting for two-stage synchronizer delay and Gray-code conversion.
- Zero data corruption under simultaneous read/write stress.

3 Synchronous FIFO Architecture

3.1 Overall Architecture

The Synchronous FIFO comprises three functional blocks operating on a single global clock:

- **Dual-Port RAM:** The storage core supporting simultaneous read and write operations via read (r_ptr) and write (w_ptr) pointers.
- **Pointer Logic:** Dual binary counters that increment on `wr_en` or `rd_en` assertions, tracking the next available memory addresses.
- **Status Logic:** A comparator block that evaluates pointer positions to generate `Full` and `Empty` flags, typically using an additional MSB to detect buffer wrap-around.

These components work in coordination to ensure data integrity, where the status logic provides essential feedback to prevent overflow or underflow during high-speed data buffering.

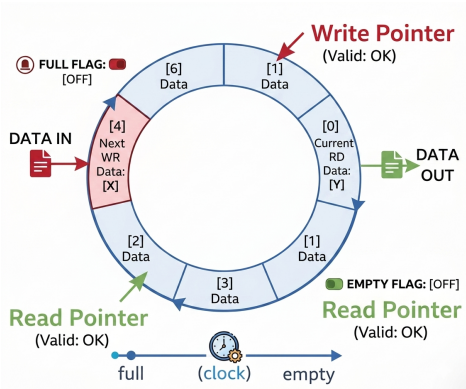

Figure 1 *Synchronous FIFO Architecture Diagram*

3.2 State Detection Strategy Based on $(N+1)$-bit Pointers

The key insight in a clean synchronous FIFO implementation is the use of an $(N{+}1)$-*bit pointer* to disambiguate the full and empty conditions:

$$\text{empty} = (W_{\text{ptr}} == R_{\text{ptr}}) \tag{1}$$

$$\text{full} = (W_{\text{ptr}}[N] \neq R_{\text{ptr}}[N]) \ \&\ (W_{\text{ptr}}[N{-}1:0] == R_{\text{ptr}}[N{-}1:0]) \tag{2}$$

where N = `ADDR_WIDTH`.

★ Design Rationale

Using an (N+1)-bit pointer can avoid the need for a separate entry counter. The MSB serves as a 'wrap-around flag': when the write operation wraps around one more time than the read operation, the address bits are equal but the MSB is different, indicating a full state. Conversely, when both the MSB and address bits are equal, it indicates that the read pointer has caught up with the write pointer, representing an empty state.

3.3 Handling Non-Power-of-Two Depth via Precise Pointer Rollover

In actual design, the depth of a FIFO is not always a power of two. Using `$clog2(DEPTH)` to define the address width introduces extra unused address space due to rounding up:

```
localparam ADDR_WIDTH = $clog2(DEPTH);
```

This may cause the pointer to access invalid memory locations if a simple binary increment is used.

To solve this, a **precise rollover mechanism** is introduced. When the pointer reaches the last valid address (`DEPTH-1`), it wraps explicitly to zero while toggling the MSB to preserve the (N+1)-bit pointer semantics:

```
if (w_pr[ADDR_WIDTH-1:0] == DEPTH - 1)
    w_pr <= {~w_pr[PR_WIDTH-1], {ADDR_WIDTH{1'b0}}};
else
    w_pr <= w_pr + 1'b1;
```

This ensures that the pointer always stays within the valid address range, while maintaining correct full/empty flag generation. Compared with power-of-two FIFOs, this approach enables arbitrary depth support without additional counters or complex control logic.

3.4 Simulation Results

Figure 2 Key waveform of Synchronous FIFO — Fill and Drain Behavior

As shown in the waveform, the FIFO is first filled until the full flag is set. Further write operations are correctly blocked to ensure that overflow does not occur. Subsequently, read operations are performed until the FIFO becomes empty, at which point the empty flag is set and further read operations are blocked. During this process, zooming in on the waveform reveals that data is read in the same order as it was written, confirming the correct behavior of the FIFO. No data corruption or boundary errors were observed.

4 Asynchronous FIFO Architecture

4.1 CDC Challenges and Metastability

Asynchronous FIFO operates across independent clock domains (wr_clk and rd_clk). The core design challenge is **Metastability** during Clock Domain Crossing (CDC): if a pointer transition violates setup/hold times of the destination flip-flop, the output may dwell at an undefined voltage.If metastability propagates, it can cause incorrect flag assertions or bring the system to a halt..

To ensure robust operation, this design implements a dual-layer protection scheme:

- **Gray Code Encoding**: Binary pointers are converted before synchronization. This reduces multi-bit switching risks to a **single-bit transition** profile. Even under timing violations, it prevents the capture of invalid intermediate states.
- **Two-Stage Synchronizers**: The Gray-coded pointer is registered twice in the destination domain. This resolves the remaining single-bit metastability, ensuring the output settles to either the **new or previous stable value**. This deterministic behavior eliminates logical noise and maximizes the `MTBF`.

4.2 Gray Code Encoding Strategy

Direct transmission of binary pointers across asynchronous clock domains introduces significant risk. For example, when transitioning from `3'b011` to `3'b100`, multiple bits toggle simultaneously, which may result in sampling an incorrect intermediate value.

To address this issue, Gray code encoding is adopted. Due to its property that only one bit changes between adjacent values, the uncertainty caused by multi-bit transitions is reduced to a single-bit metastability risk.

The binary-to-Gray code conversion is defined as:

$$G[i] = B[i] \oplus B[i+1], \quad G[N] = B[N] \tag{3}$$

In RTL implementation, this conversion can be efficiently realized using shift and XOR operations:

```
assign w_ptr_gray = (w_ptr_bin >> 1) ^ w_ptr_bin;
```

In order to correctly evaluate almost full/empty flag, the Gray code pointer must be converted

back to binary for subtraction calculations. The implementation is as:

```
function [PR_WIDTH-1:0] gray2bin;
    input [PR_WIDTH-1:0] gray;
    integer i;
    begin
        gray2bin[PR_WIDTH-1] = gray[PR_WIDTH-1];
        for (i = PR_WIDTH-2; i >= 0; i = i-1)
            gray2bin[i] = gray2bin[i+1] ^ gray[i];
    end
endfunction
```

It should be noted that the use of Gray code requires the FIFO depth to be a power of two. Only in a 2^n circular address space can the wrap-around transition (from the maximum value back to zero) preserve the single-bit change property, thereby ensuring safe CDC operation.

4.3 Two-Stage Synchronizer

Metastability cannot be eliminated in clock domain crossing, but it can be mitigated by allowing time for resolution. When a signal is sampled near a clock edge, the receiving flip-flop may enter a metastable state with an indeterminate output.

Due to the internal positive feedback of the flip-flop, this unstable state will eventually resolve to a valid logic level (0 or 1). A two-stage synchronizer leverages this property by cascading two flip-flops in the destination clock domain. The first stage may become metastable, but is given one clock cycle to settle. The second stage then samples a stable value, preventing metastability from propagating.

```
always @(posedge rd_clk or posedge reset) begin
w_ptr_gray_r1 <= w_ptr_gray;    // First stage: may become metastable
w_ptr_gray_r2 <= w_ptr_gray_r1; // Second stage: captures resolved value
end
always @(posedge wr_clk or posedge reset) begin
r_ptr_gray_w1 <= r_ptr_gray;
r_ptr_gray_w2 <= r_ptr_gray_w1;
end
```

4.4 Pessimistic Flag Assertion and Timing Safety

In an asynchronous FIFO, the FULL and EMPTY flags are determined based on read and write pointers that have been synchronized across clock domains. Specifically, the write clock domain uses the synchronized read pointer to determine the FULL condition, while the read clock domain uses the synchronized write pointer to determine the EMPTY condition.

To mitigate metastability issues caused by clock domain crossing (CDC), the pointers are first converted from binary to Gray code, and then passed through a two-stage synchronizer.

Due to the presence of synchronizers, the received pointer values may be delayed (i.e., stale values). This leads to a *pessimistic* behavior of the flag signals:

- The FULL flag may be asserted earlier than necessary (early-full)

- The EMPTY flag may be deasserted later than necessary (late-empty)

However, this pessimism is fundamentally safe, as it trades timing margin for data correctness. For example ,The EMPTY condition is determined by whether the read pointer catches up with the synchronized write pointer in the read clock domain. Due to synchronization delay, this observed write pointer always lags behind the actual one and may represent either an old or updated value.

In the worst case, the read pointer may catch up with this delayed pointer and falsely indicate EMPTY, causing the FIFO to stop reading. This is still safe, as no invalid read occurs. The system simply waits for the next synchronized update, after which normal operation resumes.

Thus, although stale pointers introduce conservative flag behavior, they never cause incorrect read or write operations, ensuring data integrity.

★ Design Rationale

CDC issues are addressed by limiting metastability rather than eliminating it. Binary pointers are converted to textbfGray code to reduce multi-bit uncertainty to a single-bit case, and a textbftwo-stage synchronizer resolves this uncertainty into a stable value. The two-stage synchronizer adds 2 cycles of latency, but this is acceptable since it guarantees the pointer value is stable before being used for flag generation.

4.5 Design Decision Matrix

The following table summarizes the key architectural decisions made during design, the alternatives considered, and the rationale for each choice:

Table 4 Design Decision Matrix

Decision	Chosen	Alternative	Rationale
Depth	Power-of-two	Arbitrary	Bit-mask wrap; Gray code friendly
Pointer width	$(N+1)$-bit	N-bit + counter	No extra counter; simpler full/empty logic
CDC method	Gray + 2-FF	Handshake	Higher throughput; single-bit transitions
Reset	Async assert, sync deassert	Fully async	Avoids reset-removal metastability

4.6 Simulation Results

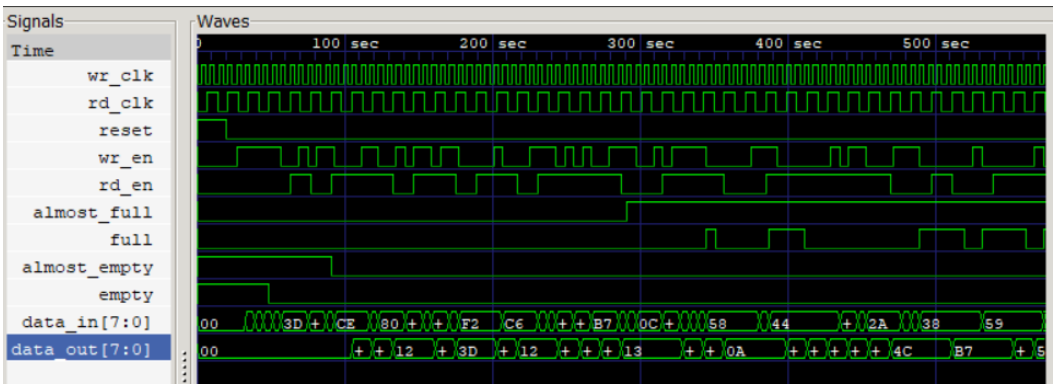

Figure 3 *Waveform of Asynchronous FIFO*

The simulation waveform demonstrates correct FIFO operation under asynchronous clock domains. Initially, data is written at a higher rate until the full flag is asserted, after which further writes are blocked to prevent overflow. The read side then consumes data at a slower clock, and the empty flag deasserts only after valid data is safely synchronized.

The waveform also shows that Gray-coded pointers enable glitch-free synchronization, with stable flag transitions. The output sequence matches the input data without errors, confirming correct FIFO behavior and reliable CDC handling.

5 Verification Methodology

5.1 Overview of Verification Strategy

The verification environment adopts a self-checking testbench architecture with an embedded **reference model** (golden model). Instead of relying on manual waveform inspection, the correctness of the DUT is automatically evaluated by comparing its output with the expected results generated by the reference model.

To improve coverage and robustness, **randomized stimulus** is applied to both read and write operations, enabling the exploration of various corner cases such as full, empty, and boundary conditions.

Additionally, lightweight coverage tracking is implemented to ensure that key states (e.g., full and empty conditions) are exercised during simulation.

5.2 Reference Model

The core challenge of FIFO verification is ensuring data arrives in the correct order and with precise timing. To address this, the testbench employs a **Reference Model**—a behaviorally equivalent but structurally independent "golden model" that runs in parallel with the DUT. By comparing the DUT's output against the model's predicted state, the verification logic is effectively **decoupled** from implementation details, ensuring the checker remains robust even if the internal architecture of the FIFO changes.

- **Synchronous FIFO Testbench**: To simplify verification logic, the reference model employs an **oversized linear array** driven by **non-wrapping integer pointers** (wr and rd). Unlike circular buffers, these pointers increment without modular reduction, ensuring **monotonic index growth** and eliminating potential index collisions or data corruption during long simulations. This approach enables status flags to be determined via **simple arithmetic comparisons** (e.g., $wr - rd \geq$ DEPTH for full), removing wrap-around ambiguity. While the pointers remain unbounded for logic tracking, the physical memory index is obtained by **truncating the lower bits**, maintaining both structural simplicity and data integrity.

```
reg [DATA_WIDTH-1:0] exp_mem [0:1024];
reg [DATA_WIDTH-1:0] expected_data_out;
integer wr = 0;
integer rd = 0;
always @(posedge clk or posedge reset) begin
if (wr_en && (wr - rd < DEPTH)) begin
    exp_mem[wr[9:0]] <= data_in;
    wr <= wr + 1;
    end
end
always @(posedge clk or posedge reset) begin
if (rd_en && wr > rd) begin
    expected_data_out <= exp_mem[rd[9:0]];
    rd <= rd + 1;
    end
end
```

- **Asynchronous FIFO Testbench**:Unlike the synchronous case, the reference model replaces the static array with a SystemVerilog **dynamic queue**, since write and read events occur on independent clock domains and cannot be coordinated through a single shared pointer pair.

```
reg [DATA_WIDTH-1:0] expected_queue[$];
// Write side
expected_queue.push_back(data_in);
// Read side
exp_data = expected_queue.pop_front();
```

  The queue's $push_back$ / pop_front semantics enforce FIFO ordering without any explicit index management, and since it is updated immediately when a transaction occurs, it always reflects the **ideal committed state** of the FIFO — serving as a reliable ground truth independent of any clock domain.

5.3 Randomized Stimulus Generation

Both testbenches apply randomized stimulus to wr_en, rd_en, and data_in on every clock edge using '*random*', each with 50% toggle probability. This approach naturally exercises concurrent read/write operations and boundary conditions such as overflow and underflow without manual test case enumeration, and is particularly effective at exposing corner-case

bugs that directed testing might overlook.

5.4 Self-checking and Timing-aware Verification

The testbench implements a fully self-checking verification flow: every read transaction is automatically compared against the reference model's prediction, with pass/fail results logged in real time via $*display*. This eliminates the need for manual waveform inspection and allows errors to be localized to the exact timestamp at which they occur.

★ Design Rationale

Timing comparison:The comparison is performed after a #1 delay to avoid race conditions between the clock edge and data updates, ensuring the Monitor captures stable post-edge values. This timing-aware strategy guarantees perfect synchronization between the DUT and reference model, effectively eliminating false mismatches caused by zero-delay artifacts.

```
@(posedge rd_clk);
if (rd_en) begin
    #1;
    if (expected_queue.size() > 0) begin
        exp_data = expected_queue.pop_front();
        if (data_out !== exp_data)
            $display("ERROR!!! Time:%0t | Expected:%h | Data:%h", $time,
                exp_data, data_out);
        else
            $display("PASS!!!! Time:%0t | Data:%h", $time, data_out);
    end
```

The scoreboard relies on `expected_queue.size()` rather than the DUT's `empty` flag to gate comparisons, preventing faulty status signals from silently masking verification checks. Additionally, it independently validates all status flags against reference thresholds, ensuring that logic errors in the flag generation are decoupled and caught separately from datapath corruptions.

5.5 Test Scenarios

Table 5 Test Scenario Coverage

Scenario	Purpose	Pass?
Sequential fill & drain	Verify basic write/read ordering	Yes
Simultaneous read & write	Verify concurrent access	Yes
Write to full, read to empty	Corner case flag assertion	Yes
Write beyond full	Verify full flag blocks excess data	Yes
Async: 200 MHz wr / 75 MHz rd	CDC functional correctness	Yes
Async: reset during transfer	Verify pointer sync after reset	Yes

6 Challenges and Debugging

6.1 Issue 1: Almost-Full False Alarm in Asynchronous FIFO Testbench

[] **Symptom**: The testbench repeatedly reported `almost_full flag errors` even when the DUT was functioning correctly.

Root Cause: The checker directly compares `expected_queue.size()` with the DUT's `almost_full` flag. The former represents the real-time state, while the latter is derived from gray-coded synchronized pointers and, due to the two-stage synchronizer, lags the actual state by 2–3 cycles, causing incorrect mismatches.

Fix: A tolerance of +3 was added to absorb the worst-case synchronization delay.

```
// BEFORE (buggy)
if ((expected_queue.size() >= (DEPTH - ALMOST_FULL_TH)) &&
    (!almost_full))
    $display("ALMOST_FULL ERROR at %0t", $time);

// AFTER (correct)
if ((expected_queue.size() >= (DEPTH - ALMOST_FULL_TH + 3)) &&
    (!almost_full))
    $display("ALMOST_FULL ERROR at %0t", $time);
```

6.2 Issue 2: Pointer Wrap-Around Error in Synchronous FIFO

[] **Symptom**: The testbench reported incorrect `full` and `empty` flags and corrupted read-back data, with the write pointer jumping to an invalid address beyond the memory range.

Root Cause: The pointer was incremented unconditionally, causing it to exceed `DEPTH-1` instead of wrapping back to `0`. Since `$clog2` rounds up to the nearest integer, the allocated address bits can represent values beyond `DEPTH-1`, so simple binary overflow does not coincide with the intended wrap point.

Fix: Add explicit wrap-around logic that toggles the MSB and clears the address bits upon reaching `DEPTH-1`.

```
localparam ADDR_WIDTH=$clog2(DEPTH);
localparam PR_WIDTH=ADDR_WIDTH+1;
// BEFORE (buggy)
w_ptr <= w_ptr + 1;

// AFTER (correct)
if (w_ptr[ADDR_WIDTH-1:0] == DEPTH - 1)
    w_ptr <= {~w_ptr[PR_WIDTH-1], {ADDR_WIDTH{1'b0}}};
else
    w_ptr <= w_ptr + 1;
```

6.3 Lessons Learned

- **Testbench checkers must account for CDC latency**: A checker that directly compares a reference model's real-time state with a DUT signal derived from synchronized pointers will produce false alarms. The 2–3 cycle synchronization delay must be tolerated in any assertion logic.
- **Explicit wrap-around is required for non-power-of-two depths**: Since `$clog2` rounds up, the address bits can represent values beyond `DEPTH-1`. Simple binary overflow does not coincide with the intended wrap point, so the pointer must be reset manually upon reaching `DEPTH-1`.
- **Gray-coded pointers must be decoded before use in subtraction**: After synchronization, Gray-coded pointers must be decoded back to binary before any subtraction. In this project, the almost_full threshold check initially operated on Gray-coded values directly, which produced incorrect results. Decoding back to binary first resolved the issue.

7 Comparative Analysis: Sync vs. Async FIFO

Table 6 Synchronous vs. Asynchronous FIFO — Design Comparison

Criterion	Synchronous FIFO	Asynchronous FIFO
Clock domains	Single	Dual (independent)
Pointer logic	Binary $(N+1)$-bit	Binary $\rightarrow$ Gray + 2-FF sync
Flag generation	Direct binary comparison	Synchronized Gray comparison
Latency	0 cycles (same domain)	2 read cycles (CDC latency)
Complexity	Low	Moderate
Use case	Same-frequency data buffering	CDC, SoC interface, SerDes

8 Conclusion

This project successfully implemented and verified two FIFO architectures, progressing from a clean synchronous design to a metastability-safe asynchronous variant. The $(N+1)$-bit pointer scheme and Gray-code CDC strategy were validated through comprehensive simulation.

Key takeaways:

- Both designs passed all functional tests, including corner cases such as simultaneous read/write, back-to-back full/empty transitions, and almost-full/empty threshold

behavior.

- The asynchronous FIFO met its correctness targets after resolving synchronization-related issues; the two-stage Gray-code synchronizer effectively suppressed metastability across independent clock domains.
- This project reinforced that CDC design requires disciplined pointer management—Gray code must be used for transport and decoded before arithmetic—and that testbench checkers must account for synchronization latency to avoid false alarms.

9 Future Improvements

- **Non-Power-of-Two Depth**: Generalize the pointer wrap logic using modulo arithmetic to support arbitrary FIFO depths, enabling more flexible resource utilization.
- **Adaptive Flow Control**: Leverage the existing `almost_full`/`almost_empty` flag to dynamically throttle the write rate at runtime, rather than relying on static backpressure. This enables finer-grained throughput regulation and reduces the risk of burst-induced overflow.
- **ECC Protection**: Add single-error-correct, double-error-detect (SECDED) parity to the memory array, enhancing reliability for safety-critical or high-availability applications.